AI Project

Project idea : Handwritten digit recognition

Algorithms :

1- Decision Tree : it selects the best attribute to split the data based on a metric such as entropy or Gini impurity.

```
In [7]: clf = DecisionTreeClassifier(criterion="entropy",random_state=100, max_depth=10)
        clf.fit(X_train_flattened, y_train)
        clf.score(X_test_flattened, y_test)

Out[7]: 0.8727
```

2- Convolution Neural Network : CNN can have tens or hundreds of layers that each learn to detect different features of an image. Filters are applied to each training image at different resolutions, and the output of each convolved image is used as the input to the next layer.

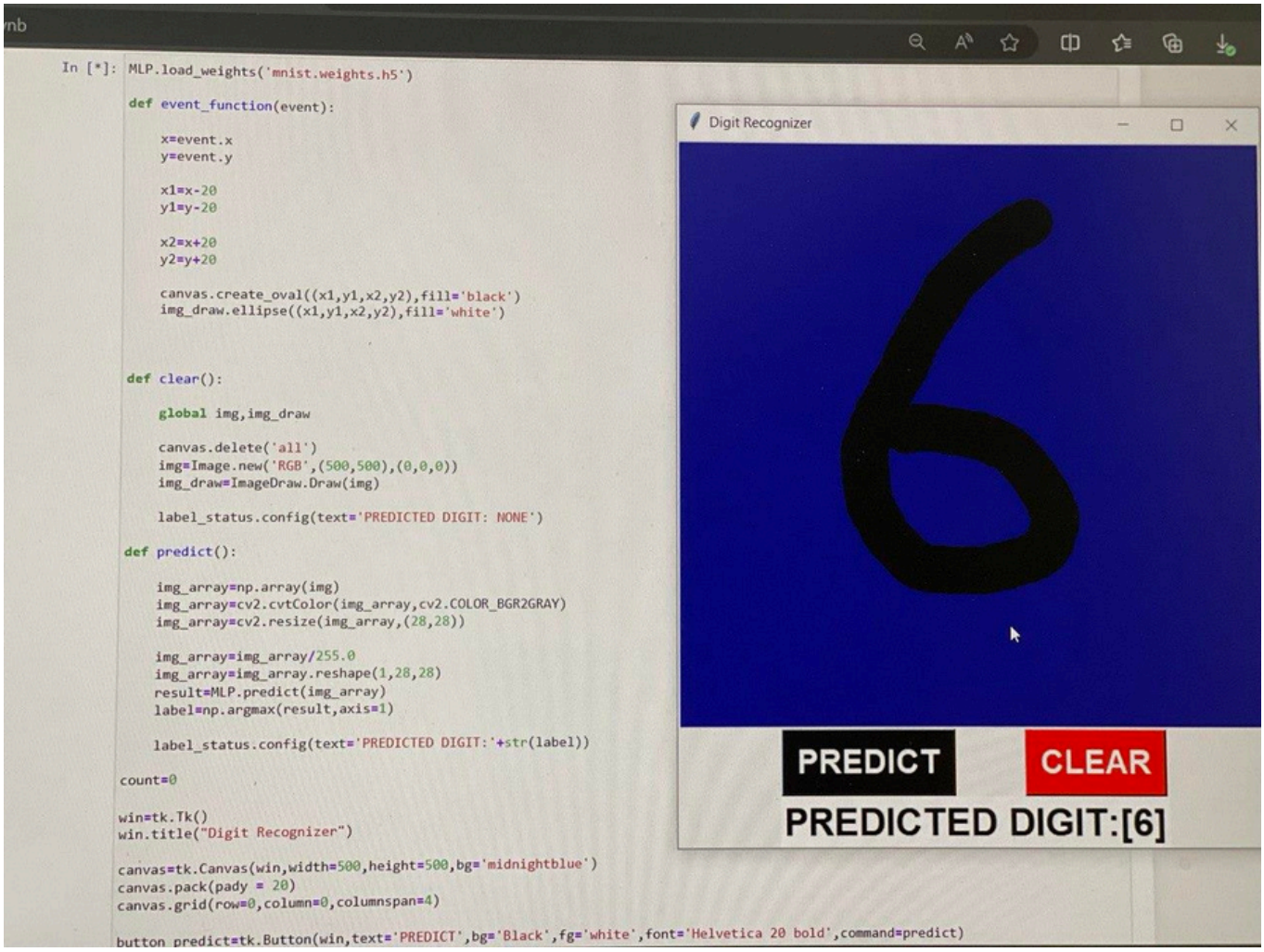
```
In [22]: CNN = Sequential()

         CNN.add (Conv2D(64,(3,3), input_shape = input_shape))
         CNN.add(Activation("relu"))
         CNN.add (MaxPooling2D(pool_size=(2,2)))


         CNN.add (Conv2D(64,(3,3)))
         CNN.add(Activation("relu"))
         CNN.add (MaxPooling2D(pool_size=(2,2)))


         CNN.add (Conv2D(64,(3,3)))
         CNN.add(Activation("relu"))
         CNN.add (MaxPooling2D(pool_size=(2,2)))


         CNN.add (Flatten())
         CNN.add(Dense(64))
         CNN.add(Activation("relu"))


         CNN.add(Dense(32))
         CNN.add(Activation("relu"))


         CNN.add(Dense(10))
         CNN.add(Activation("softmax"))


         CNN.compile(loss='sparse_categorical_crossentropy',optimizer='adam',metrics=['accuracy'])

         CNN.fit(x_train, y_train, epochs=7)

         Epoch 1/7
         C:\ANACONDA\lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input
         _dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model
         instead.
           super().__init__(activity_regularizer=activity_regularizer, **kwargs)
         1875/1875 ━━━━━━━━━━━━━━━━━━━━ 14s 7ms/step - accuracy: 0.8518 - loss: 0.4689
         Epoch 2/7
         1875/1875 ━━━━━━━━━━━━━━━━━━━━ 14s 8ms/step - accuracy: 0.9746 - loss: 0.0823
         Epoch 3/7
         1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.9837 - loss: 0.0522
         Epoch 4/7
         1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.9861 - loss: 0.0438
         Epoch 5/7
         1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9896 - loss: 0.0343
         Epoch 6/7
         1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 7ms/step - accuracy: 0.9908 - loss: 0.0286
         Epoch 7/7
         1875/1875 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step - accuracy: 0.9933 - loss: 0.0211

Out[22]: <keras.src.callbacks.history.History at 0x19ee54e3b50>
```

3- Multi layer perceptron : have an input and output layers, and one or more hidden layers with many neurons stacked together. And while in the Perceptron the neuron must have an activation function like Relu , sigmoid .

```
In [43]: model=Sequential()

         model.add(Flatten(input_shape=(28,28)))

         model.add(Dense(512,activation='relu'))
         model.add(Dense(256,activation='relu'))
         model.add(Dense(128,activation='sigmoid'))
         model.add(Dense(64,activation='sigmoid'))
         model.add(Dense(10,activation='softmax'))

         model.compile(loss='sparse_categorical_crossentropy',optimizer='adam',metrics=['accuracy'])

         model.fit(X_train, y_train, epochs=5)

         model.evaluate(X_test, y_test)

         #model.save_weights('C:\handwriting_digit_recognition/mnist.weights.h5')
         model.load_weights('mnist.weights.h5')
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8628 - loss: 0.5579
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9725 - loss: 0.0954
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9818 - loss: 0.0598
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9871 - loss: 0.0441
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9886 - loss: 0.0371
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9766 - loss: 0.0884
```

4- Principal component analysis(PCA) : is a dimensionality reduction method that is often used to reduce the dimensionality of large data sets, by transforming a large set of variables into a smaller one that still contains most of the information in the large set.

```
In [28]: pca = PCA(0.95)
         X_train_flattened_pca = pca.fit_transform(X_train_flattened)
         X_test_flattened_pca = pca.transform(X_test_flattened)
         X_train_flattened_pca.shape , X_test_flattened_pca.shape

Out[28]: ((60000, 154), (10000, 154))
```

5- Graphical user interface (GUI) : that allows users to draw a digit and predict it